Online Summer Training Course

On

MERN Stark Development with a Capstone

Submitted by

Vedant Singh Rajput

Registration No: 12209007

Btech CSE (3rd Year)

Under the Guidance of

Anoop Garg sir

School of Computer Science & Engineering

Lovely Professional University, Phagwara

DECLARATION

I now declare that I have completed my Online Summer Training Course on MERN Stark Development with a Capstone from programming pathshala from June 05, 2024, to August 05, 2024, under the guidance of the Human Resource Development Center. I declare that I have worked with full dedication during these 2 months of training, and my learning outcomes fulfil the requirements for the course.

Date: 20 July 2024

Name of Student: Vedant Singh Rajput

Registration No: 12209007

ACKNOWLEDGMENT

I, Vedant Singh Rajput hereby acknowledge the successful completion of the Online Summer Training Course on MERN Stack Development, conducted from June 05, 2024 to August 05, 2024.This course was organized by programming pathshala and included a capstone project that allowed me to apply my learning to real-world scenarios.

I sincerely appreciate the guidance and support provided by my mentors and instructors throughout this training. The comprehensive curriculum and hands-on experience have greatly enhanced my understanding of MERN Stack technologies and their application in full-stack development.

This acknowledgment confirms my commitment to continuing to apply the skills and knowledge gained during this training in future projects and professional endeavors.

Date: 6 August 2024
Name: Vedant Singh Rajput

Certificate

Programming Pathshala
669ba48775386515a8255982

Summer Training Program

Vedant Singh Rajput

has successfully completed course on

MERN Stack Development with a Capstone Project

A Program offered by Programming Pathshala, to be excellent Software Engineers.

Programming Pathshala

2024-07-20
DATE

programmingpathshala.com

Anoop Garg
Co-founder, Programming Pathshala

Kindly verify Programming Pathshala's document on Blockchain by scanning the QR code.
https://certificates.programmingpathshala.com/docs/76c70943a8bcbe033ac0d0df5b4cd7cd7f860ab9e154782c472d1cf1b20c1dc4

Introduction

Web development encompasses the process of creating websites and web applications for the Internet. It spans a wide spectrum, from developing simple static pages to crafting complex, dynamic web-based applications, electronic businesses, and social networks.

Web development is broadly categorized into two divisions: front-end development (client-side) and back-end development (server-side).

Front-end development focuses on constructing the visual elements that users interact with when they load a web application. This involves using HTML, CSS, and JavaScript. These technologies work together to create the content, design, and interactive features of the web.

- **HTML (HyperText Markup Language)** provides the basic structure of websites, enabling developers to format content and define the layout.
- **CSS (Cascading Style Sheets)** controls the visual styling, including layout, colors, and fonts, giving web applications their unique look and feel.
- **JavaScript** adds interactivity and dynamic behavior to web pages, allowing for responsive user interfaces and enhanced user experiences.

Back-end development involves managing the server-side operations that power the front-end. This typically includes database management, server logic, and API development, which are crucial for handling data, processing requests, and generating dynamic content.

During my summer training at Programming Pathshala, I focused on mastering the MERN stack—a powerful and popular technology stack used for full-stack web development. The MERN stack comprises four key technologies:

- **MongoDB:** A NoSQL database used for storing and managing data in a flexible, scalable way.
- **Express.js:** A web application framework for Node.js that simplifies the process of building robust server-side applications.
- **React.js:** A JavaScript library for building user interfaces, particularly single-page applications, where responsiveness and performance are critical.
- **Node.js:** A JavaScript runtime that allows developers to execute JavaScript code on the server side, facilitating the creation of scalable network applications.

Technology Learnt

- HTML and CSS
- Javascript
- Node.js
- MongoDB
- React.js
- Express.js
- Deployment and Hosting

Basic Device Requirements

1. Hardware Requirements:

- **Processor:** A multi-core processor is recommended, such as Intel Core i5 or i7, AMD Ryzen 5 or 7, or equivalent.
- **RAM:** At least 8GB of RAM is recommended.
- **Storage:** A Solid State Drive (SSD) with at least 256GB of storage is recommended. SSDs provide faster read/write speeds compared to traditional Hard Disk Drives (HDDs), significantly reducing the time required for tasks like booting up the system, loading projects, or compiling code.
- **Graphics:** Integrated graphics are usually sufficient for MERN stack development.

2. Software Requirements:

- **Operating System:** MERN stack development is supported on Windows, macOS, and Linux.
- **Text Editor/IDE:** A good text editor or Integrated Development Environment (IDE) is essential. Popular choices include Visual Studio Code, Sublime Text, and Atom.

3.Additional Considerations:

- **Internet Connection:** A reliable internet connection is important for downloading dependencies, using cloud services, and accessing online resources and documentation.

HTML (HyperText Markup Language)

HTML, or HyperText Markup Language, is the standard markup language used for creating web pages. It provides the structure for web content, allowing developers to format text, images, links, and other multimedia elements within a web document. HTML is foundational to web development, as it defines the content and layout of a webpage.

Advantages of HTML:

- **Simplicity:** HTML is easy to learn and use, making it accessible for beginners in web development.
- **Structured Content:** It provides a clear structure for content, facilitating the organization of text, images, and other media elements.
- **Cross-Platform Compatibility:** HTML works across all web browsers and devices, ensuring a consistent user experience.
- **Search Engine Friendly:** HTML documents are easily indexed by search engines, improving search engine optimization (SEO) and visibility.
- **Integration with CSS and JavaScript:** HTML can be seamlessly integrated with CSS for styling and JavaScript for interactivity, enhancing the functionality of web applications.
- **Open Standard:** HTML is an open standard maintained by the World Wide Web Consortium (W3C), ensuring ongoing support and development.

Disadvantages of HTML:

- **Static Nature:** HTML is primarily static; without additional technologies like CSS and JavaScript, it lacks interactivity and dynamic content.
- **Limited Functionality:** HTML alone cannot handle complex tasks such as form validation or data manipulation, which require additional scripting languages.
- **Dependency on Browser Rendering:** The appearance of HTML content can vary between different web browsers, necessitating careful testing and optimization.
- **Lack of Control over Design:** While HTML provides structure, it does not offer extensive design capabilities, making CSS essential for advanced styling.
- **No Built-in Security Features:** HTML does not provide security features, making web applications vulnerable without proper security measures in place.

CSS (Cascading Style Sheets)

CSS, or Cascading Style Sheets, is a style sheet language used for describing the presentation and formatting of HTML documents. CSS allows developers to separate content from design, enabling them to control the layout, colors, fonts, and overall visual appearance of web pages. This separation enhances maintainability and improves the user experience.

Advantages of CSS:

- **Separation of Content and Presentation:** CSS allows developers to separate the content (HTML) from its presentation, making it easier to manage and update designs without affecting the content.
- **Improved Load Times:** By using external CSS files, web pages can load faster since styles are cached by browsers, reducing the need to reload styles for every page.
- **Consistent Styling:** CSS enables developers to apply consistent styles across multiple pages, ensuring a uniform look and feel throughout a website.
- **Responsive Design:** CSS supports media queries, allowing developers to create responsive layouts that adapt to different screen sizes and devices.
- **Ease of Maintenance:** Updating styles is straightforward; changes made in the CSS file automatically reflect across all pages that use the styles, simplifying maintenance.
- **Advanced Styling Features:** CSS provides advanced features such as animations, transitions, and flexbox/grid layouts, allowing for dynamic and modern designs.

Disadvantages of CSS:

- **Browser Compatibility Issues:** Different browsers may interpret CSS styles differently, requiring careful testing and adjustments to ensure consistent rendering.
- **Learning Curve:** While CSS is generally easier to learn than programming languages, mastering its intricacies (such as specificity, cascading, and box model) can be challenging.
- **Complexity in Large Projects:** In large-scale projects, managing and organizing CSS can become complicated, leading to difficulties in maintaining styles.
- **Limited Functionality:** CSS is primarily for presentation; it cannot handle dynamic behavior or complex interactivity, which require JavaScript or other scripting languages.
- **Performance Overhead:** Excessive use of CSS, especially with complex selectors and styles, can impact rendering performance, leading to slower page loads.

JavaScript

JavaScript is a high-level, interpreted programming language that is primarily used for creating dynamic and interactive web applications. As a core technology of the web, alongside HTML and CSS, JavaScript allows developers to enhance user interfaces and implement complex features on web pages. It enables client-side scripting, meaning scripts are executed on the user's browser, which allows for real-time interaction without needing to reload the page.

Advantages of JavaScript:

- **Interactivity:** JavaScript enables developers to create interactive elements on web pages, such as forms, animations, and games, enhancing user engagement.
- **Cross-Platform Compatibility:** JavaScript runs on all major web browsers and platforms, ensuring a consistent experience across different devices.
- **Asynchronous Programming:** With features like callbacks, promises, and async/await, JavaScript allows for non-blocking operations, improving performance and user experience.
- **Rich Ecosystem:** JavaScript has a vast ecosystem of libraries and frameworks (e.g., React, Angular, Vue.js) that streamline development and enhance functionality.
- **Full-Stack Development:** JavaScript can be used on both the client-side (browser) and server-side (with Node.js), enabling full-stack development with a single language.
- **Community Support:** JavaScript has a large and active community, providing extensive resources, documentation, and support for developers at all levels.

Disadvantages of JavaScript:

- **Browser Dependency:** JavaScript execution can vary across different browsers, leading to compatibility issues and requiring thorough testing.
- **Security Vulnerabilities:** Being a client-side language, JavaScript is susceptible to various security risks, such as cross-site scripting (XSS) and code injection attacks.
- **Performance Issues:** Poorly written JavaScript can lead to performance bottlenecks, causing slow load times and unresponsive user interfaces.
- **Complexity in Larger Applications:** Managing and organizing JavaScript code in large applications can become complex, leading to challenges in maintainability and scalability.
- **Limited Multithreading:** JavaScript is single-threaded, which can be a limitation for performance in CPU-intensive tasks, although web workers provide a workaround for background processing.

Node.js

Node.js is an open-source, cross-platform JavaScript runtime environment that allows developers to execute JavaScript code on the server side. Built on Chrome's V8 JavaScript engine, Node.js enables the development of scalable and high-performance applications, particularly for web servers and real-time applications. It utilizes an event-driven, non-blocking I/O model, making it efficient for handling concurrent connections.

Advantages of Node.js:

- **Asynchronous and Event-Driven:** Node.js uses a non-blocking I/O model, allowing it to handle multiple connections simultaneously without waiting for any single operation to complete, resulting in high performance and responsiveness.
- **Single Language for Full Stack Development:** With Node.js, developers can use JavaScript for both client-side and server-side programming, simplifying the development process and improving collaboration.
- **Fast Execution:** Built on the V8 engine, Node.js compiles JavaScript to native machine code, resulting in fast execution and reduced latency for applications.
- **Rich Ecosystem:** Node.js has a vast library of packages available through npm (Node Package Manager), enabling developers to easily integrate third-party modules and tools into their applications.
- **Scalability:** Node.js is designed for scalability, making it suitable for building large-scale applications that can handle high traffic loads and real-time data processing.
- **Active Community:** Node.js has a large and active community, providing extensive resources, documentation, and support, which fosters rapid development and innovation.

Disadvantages of Node.js:

- **Callback Hell:** The asynchronous nature of Node.js can lead to complex nested callbacks, making code harder to read and maintain (though this can be mitigated with Promises and async/await).
- **Not Suitable for CPU-Intensive Tasks:** Node.js is best suited for I/O-bound applications. CPU-intensive tasks can block the event loop, leading to performance bottlenecks.
- **Security Risks:** As with any server-side technology, Node.js applications can be vulnerable to security issues such as denial-of-service attacks and code injection if not properly managed.
- **Steep Learning Curve:** For developers unfamiliar with asynchronous programming or the JavaScript ecosystem, the learning curve can be challenging.
- **Limited Built-in Functionality:** Node.js provides a minimal set of built-in libraries, meaning developers often need to rely on third-party modules, which can introduce additional dependencies and complexities.

Express.js

Express.js is a minimal and flexible Node.js web application framework that provides a robust set of features for building web and mobile applications. It simplifies the process of creating server-side applications and APIs by offering a range of tools and conventions that facilitate routing, middleware integration, and HTTP request handling.

Advantages of Express.js:

- **Simplicity and Minimalism:** Express.js is lightweight and straightforward, making it easy to learn and use, especially for developers familiar with Node.js.
- **Middleware Support:** Express allows the use of middleware functions to handle requests, responses, and error handling, enabling modular code organization and better control over request processing.
- **Robust Routing:** The framework provides a powerful routing mechanism that allows developers to define routes for different HTTP methods and URL paths, facilitating the creation of RESTful APIs.
- **Flexibility:** Express is unopinionated, meaning developers have the freedom to structure their applications as they see fit, allowing for custom solutions tailored to specific project needs.
- **Integration with Template Engines:** Express supports various template engines (e.g., Pug, EJS), enabling developers to render dynamic HTML content easily.
- **Community and Ecosystem:** A large and active community supports Express.js, providing numerous resources, middleware, and plugins that enhance its functionality.

Disadvantages of Express.js:

- **Minimal Documentation:** While the core features are well-documented, the extensive use of middleware can lead to challenges in finding documentation for specific use cases or third-party modules.
- **No Built-in Security Features:** Express.js does not come with built-in security features, so developers must implement security measures, such as input validation and authentication, to protect their applications.
- **Learning Curve for Beginners:** While Express itself is simple, understanding middleware and the overall structure of a Node.js application can be challenging for beginners.
- **Callback Hell:** Due to its asynchronous nature, Express applications can experience "callback hell," making the code harder to read and maintain; however, this can be mitigated with Promises or async/await.
- **Performance Overhead:** For very high-performance applications, the additional layer of Express may introduce some performance overhead compared to using pure Node.j

Reasons for Choosing the MERN Stack

I chose the MERN stack because I love the thrill of seeing my imagination come to life. The MERN stack—MongoDB, Express.js, React, and Node.js—allows me to build powerful web applications that resonate with my vision. Here are a few reasons why I opted for this technology:

1. **In-Demand Skills:** MERN stack developers are highly sought after in the job market. Whether I work for an organization or as a freelancer, my skills in the MERN stack will always be in demand as businesses seek to build robust web applications.
2. **Lucrative Career Opportunities:** The MERN stack opens doors to well-paying job opportunities. The combination of these technologies enables the creation of dynamic applications, making it a valuable skill set that companies are willing to invest in.
3. **Flexibility:** With the MERN stack, I can work from anywhere, including home. This flexibility is essential in today's work environment, allowing me to balance my personal and professional life while developing innovative applications.
4. **Creativity:** MERN stack development is all about creativity and problem-solving. I can use my skills to build visually appealing and user-friendly applications that enhance user experience and engagement.
5. **Diverse Career Options:** The online presence is crucial for all sectors, and MERN stack development allows me to create successful applications for various industries. Whether I choose to work for corporations, non-profits, or as a self-employed freelancer, I can make a significant impact.
6. **Constantly Evolving Field:** The MERN stack is always evolving, offering exciting opportunities to learn new tools and technologies. This dynamic environment keeps me engaged and motivated to continuously improve my skills

Learning Outcomes from MERN Stack Training

Through my training in the MERN stack, I have gained valuable skills and knowledge that enhance my web development capabilities. Here are the key learning outcomes:

1. **Proficient in MongoDB:**
    - Developed a solid understanding of NoSQL databases and how to effectively use MongoDB for data storage and retrieval.
    - Learned to design and manage databases, implement data models, and perform CRUD operations.
2. **Mastered Express.js:**
    - Gained expertise in building RESTful APIs and web applications using Express.js, allowing for efficient server-side logic and routing.
    - Understood middleware integration for handling requests, responses, and error management.
3. **Skilled in React:**
    - Acquired skills in building dynamic user interfaces with React, including component-based architecture and state management.
    - Learned to implement hooks, context API, and lifecycle methods to enhance application functionality and user experience.
4. **Competent in Node.js:**
    - Developed a strong foundation in server-side JavaScript programming with Node.js, enabling the creation of scalable web applications.
    - Understood asynchronous programming concepts and event-driven architecture for improved performance.
5. **Full-Stack Development Capabilities:**
    - Gained the ability to develop complete web applications from front-end to back-end, providing a comprehensive understanding of the development process.
    - Learned to integrate front-end and back-end components effectively, ensuring seamless data flow and user interaction.
6. **Version Control with Git:**
    - Enhanced my understanding of version control systems, particularly Git, for managing code changes and collaborating with other developers.
    - Learned to use platforms like GitHub for code hosting and project management.
7. **Real-World Project Experience:**

- Engaged in hands-on projects that simulated real-world scenarios, applying theoretical knowledge to practical situations.
- Developed problem-solving skills and the ability to debug and optimize code effectively.

8. **Adaptability and Continuous Learning:**
    - Cultivated a mindset of adaptability in the ever-evolving field of web development, recognizing the importance of staying updated with new technologies and practices.
    - Developed a passion for continuous learning, encouraging exploration of advanced topics and emerging trends in web development.

Weekly Training Overview for MERN Stack

Date	**Topic**	**Section**
05/06/24	Orientation	Orientation
08/06/24	HTML Basics	HTML
09/06/24	HTML	HTML
15/06/24	CSS Basics	CSS
16/06/24	CSS	CSS
22/06/24	CSS and JS Intro, GitHub Pages	CSS & JavaScript
23/06/24	Mutability, Loose & Strict Equality, Function in Multiple Ways (Part 1)	JavaScript
29/06/24	Object and Array (Part 2)	JavaScript
30/06/24	JS Objects, Arrays, DOM API	JavaScript
06/07/24	Events, Form Handling, Callbacks	JavaScript
07/07/24	Asynchronous Callbacks, fetch()	JavaScript
20/07/24	Promises and Dynamic Rendering	Node.js
21/07/24	Backend Intro, Node.js, Modules (Export/Import), FS, Sync - Async (CB and PR)	Node.js
28/07/24	FS, HTTP, URL, Server-Side Rendering of Webpage	Express.js
03/08/24	SSR vs CSR, Express.js, REST Architecture, Postman, GET, POST	Express.js & MongoDB
04/08/24	CRUD, Middlewares, Refactoring as per MVC, Router, Controller, Own Middleware, MongoDB Intro	MongoDB
10/08/24	MongoDB and Mongoose Intro, dotenv, Cluster, Collections, BSON/Documents, CRUD Operations in MongoDB using Mongoose, Middleware Chaining	MongoDB

11/08/24	Advanced Querying with List API (Mongoose, Express.js)	MongoDB
17/08/24	Basics of React (CDN, React.createElement, JSX, Babel), React Element, React Component	React
18/08/24	Installation of React and Parcel (Bundler), Understanding Client-Server Connection for React App, Export - Import Custom Components, CSS Styling, Props and Destructuring, Looping: forEach vs map in React	React
24/08/24	Props Destructuring and Default Value, Routing (Router, useNavigate, Link vs a), Key Property, Optional Chaining	React
	Component Re-Rendering and State, useState Hook	Full Stack
	Script, Routing, useState, useEffect, Link vs Anchor Tag	Full Stack
	useParams, useNavigate, Context API	
	Frontend-Backend Connection, Signup, bcrypt, Schema.Pre, Schema.Methods	

Building a MERN stack application involves using MongoDB, Express.js, React, and Node.js. A capstone project in this context typically involves creating a full-fledged web application that showcases all the skills you've learned in using these technologies.

MERN Stack Overview

- **MongoDB: A NoSQL database that stores data in JSON-like documents.**
- **Express.js: A web application framework for Node.js, used to build web APIs.**
- **React: A JavaScript library for building user interfaces, particularly single-page applications.**
- **Node.js: A JavaScript runtime that allows you to run JavaScript on the server side.**

Capstone Project: Full-Stack MERN Application

Project Idea: Task Management Application

Features:

1. **User Authentication: Users can sign up, log in, and log out.**
2. **Task Management: Users can create, read, update, and delete tasks.**
3. **Categories/Tags: Users can categorize tasks and filter them.**
4. **Priority Levels: Assign priorities to tasks.**
5. **Due Dates and Reminders: Set due dates and get reminders.**
6. **Responsive Design: Application should work on both desktop and mobile devices.**

Frontend: React

1. **Create a React App:**
    - **Initialize a new React application using Create React App:**

bash

Copy code

```
npx create-react-app task-manager
cd task-manager
```

- **Install necessary dependencies:**

bash

Copy code

```
npm install axios react-router-dom
```

2. **Components:**
    - **Authentication Components: Login, Register, Logout**

- **Task Components: TaskList, TaskItem, TaskForm**
- **Navigation Components: Navbar, Footer**

3. React Router: Set up routing for different pages:

jsx

Copy code

```
import React from 'react';
import { BrowserRouter as Router, Route, Switch } from 'react-router-dom';
import Home from './components/Home';
import Login from './components/Login';
import Register from './components/Register';
import TaskList from './components/TaskList';

function App() {
  return (
    <Router>
      <div className="App">
        <Navbar />
        <Switch>
          <Route path="/" exact component={Home} />
          <Route path="/login" component={Login} />
          <Route path="/register" component={Register} />
          <Route path="/tasks" component={TaskList} />
        </Switch>
      </div>
    </Router>
  );
}

export default App;
```

4. State Management: Use useState and useEffect hooks for state management. Optionally, consider using a state management library like Redux if your app grows in complexity.
5. API Integration: Use axios to make HTTP requests to the backend API.

Backend: Node.js, Express.js, MongoDB

1. **Set Up Express Server:**
    - **Initialize a new Node.js project:**

bash

Copy code

```
mkdir server
cd server
npm init -y
npm install express mongoose bcryptjs jsonwebtoken cors
```

2. **Create an Express Server:**
    - **Set up a basic Express server:**

js

Copy code

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');

const app = express();
app.use(express.json());
app.use(cors());

const PORT = process.env.PORT || 5000;

mongoose.connect('mongodb://localhost:27017/task-manager', {
  useNewUrlParser: true,
  useUnifiedTopology: true,
}).then(() => {
  console.log('MongoDB connected');
}).catch(err => console.log(err));

app.listen(PORT, () => {
```

```
  console.log(`Server running on port ${PORT}`);
});
```

3. Models:
   - User Model: Schema for user details and hashed passwords.
   - Task Model: Schema for tasks with fields for description, due date, category, etc.
4. Routes:
   - Auth Routes: /api/auth/register, /api/auth/login, /api/auth/logout
   - Task Routes: /api/tasks, /api/tasks/:id
5. Middleware:
   - Auth Middleware: For protecting routes and verifying JWT tokens.
6. JWT Authentication:
   - Implement user authentication using jsonwebtoken for token-based authentication.

Deployment

1. Frontend Deployment: Host the React app on platforms like Netlify or Vercel.
2. Backend Deployment: Deploy the Node.js application on platforms like Heroku or Render.
3. Database: Host MongoDB on a cloud database provider like MongoDB Atlas.

Example Code Snippets

- Register Route:

js

Copy code

```
const express = require('express');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const User = require('../models/User');

const router = express.Router();

router.post('/register', async (req, res) => {
```

```
    const { username, email, password } = req.body;
    try {
        const salt = await bcrypt.genSalt(10);
        const hashedPassword = await bcrypt.hash(password, salt);

        const newUser = new User({ username, email, password: hashedPassword });
        await newUser.save();

        const token = jwt.sign({ id: newUser._id }, 'secret', { expiresIn: '1h' });
        res.json({ token });
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
});

module.exports = router;
```

- **Task Model:**

js

Copy code

```
const mongoose = require('mongoose');

const TaskSchema = new mongoose.Schema({
  userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
  title: {
    type: String,
    required: true
  },
  description: {
    type: String,
    required: true
  },
  dueDate: {
    type: Date,
    required: true
  },
  priority: {
    type: String,
    enum: ['Low', 'Medium', 'High'],
    default: 'Medium'
  },
  completed: {
    type: Boolean,
```

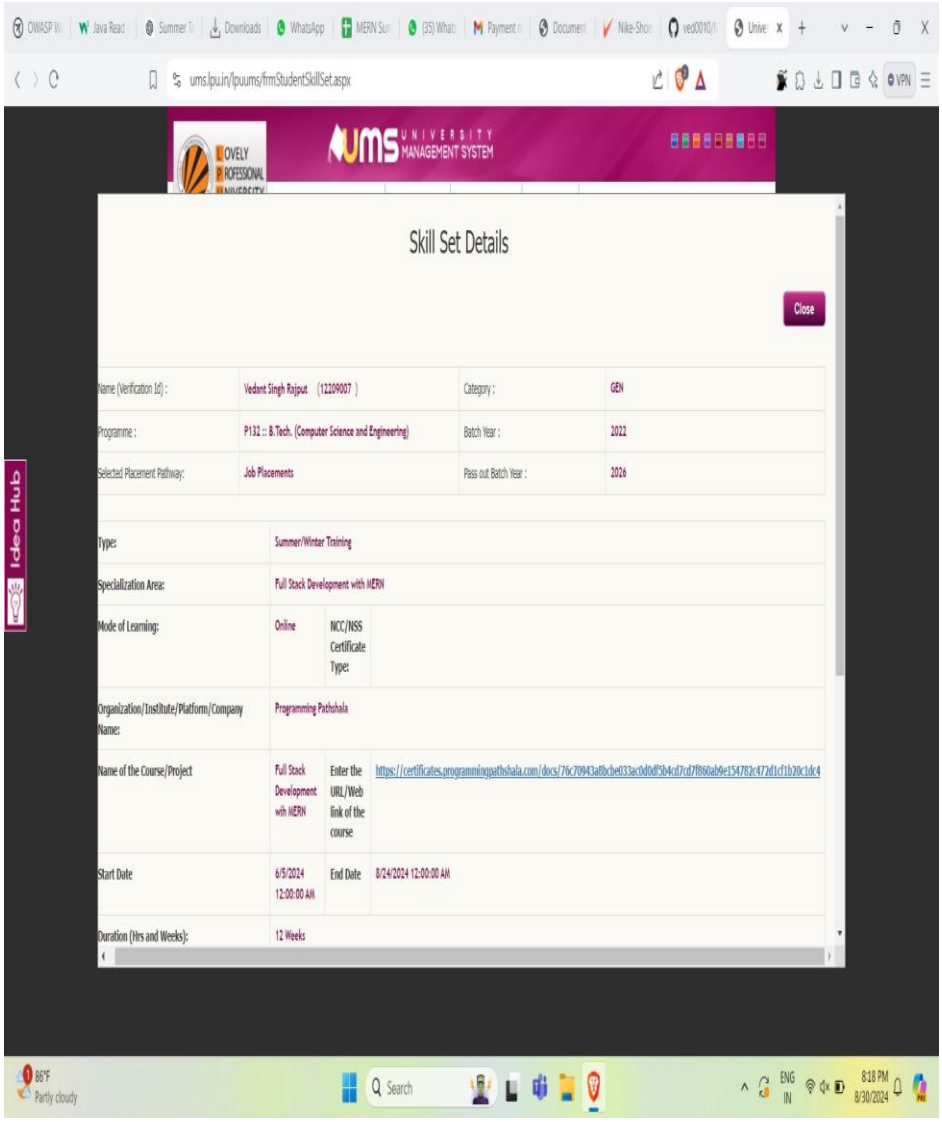
```
        default: false
    }
});


module.exports = mongoose.model('Task', TaskSchema);
```

This is a high-level overview to get you started. As you develop your application, you'll likely need to add more features, optimize the code, and enhance the UI/UX. If you have any specific questions or need further details on any part of the stack, feel free to ask!

Bibliography

1. **Web Development Basics**
   - Mozilla Developer Network (MDN). (n.d.). *Introduction to web development*. Retrieved from https://developer.mozilla.org/en-US/docs/Learn/Getting_started_with_the_web
2. **HTML**
   - W3Schools. (n.d.). *HTML Introduction*. Retrieved from https://www.w3schools.com/html/html_intro.asp
3. **CSS**
   - W3Schools. (n.d.). *CSS Introduction*. Retrieved from https://www.w3schools.com/css/css_intro.asp
4. **JavaScript**
   - Eloquent JavaScript. (n.d.). *Introduction*. Retrieved from https://eloquentjavascript.net/
5. **Node.js**
   - Node.js Foundation. (n.d.). *About Node.js*. Retrieved from https://nodejs.org/en/about/
6. **Express.js**
   - Express. (n.d.). *Express - Node.js web application framework*. Retrieved from https://expressjs.com/
7. **MongoDB**
   - MongoDB Inc. (n.d.). *MongoDB Documentation*. Retrieved from https://docs.mongodb.com/
8. **React**
   - React. (n.d.). *React - A JavaScript library for building user interfaces*. Retrieved from https://reactjs.org/
9. **Full Stack Development**
   - freeCodeCamp. (n.d.). *Full Stack Development*. Retrieved from https://www.freecodecamp.org/learn/
10. **MERN Stack**
    - MERN.io. (n.d.). *MERN Stack: MongoDB, Express.js, React.js, and Node.js*. Retrieved from https://mern.io/


ums.lpu.in/lpuums/frmStudentSkillSet.aspx
UMS UNIVERSITY MANAGEMENT SYSTEM
Skill Set Details
Close
Name (Verification Id) : Vedant Singh Rajput (12209007)
Category : GEN
Programme : P132 :: B.Tech. (Computer Science and Engineering)
Batch Year : 2022
Selected Placement Pathway: Job Placements
Pass out Batch Year : 2026
Type: Summer/Winter Training
Specialization Area: Full Stack Development with MERN
Mode of Learning: Online
NCC/NSS Certificate Type:
Organization/Institute/Platform/Company Name: Programming Pathshala
Name of the Course/Project Full Stack Development with MERN
Enter the URL/Web link of the course
https://certificates.programmingpathshala.com/docs/76c70943a8bcbe033ac0d0df5b4cd7cd7f860ab9e154782c472d1cf1b20c1dc4
Start Date 6/5/2024 12:00:00 AM
End Date 8/24/2024 12:00:00 AM
Duration (Hrs and Weeks): 12 Weeks
Idea Hub
Search
ENG IN
8:18 PM 8/30/2024